JSPM's
RAJARSHI SHAHU COLLEGE OF ENGINEERING
TATHAWADE, PUNE-33

DEPARTMENT OF COMPUTER ENGINEERING

Mini Project

on

"SMS Spam Classifier"

-Submitted by-

Yuvraj Kadam (RBT24CS150)

Roll No.: 09

Class and Division: TY Div C-1

Course: Data Mining and Analytics (DMA)

Academic Year 2026-27

(Semester-I)

1. **Title**
2. **Abstract**
3. **Introduction**
4. **Problem Statement**
5. **Objectives**
6. **Data set and Libraries required**
7. **Approach/ Methodology**
8. **Implementation**
9. **Results**
10. **Conclusion**

1. Title

End-to-End SMS Spam Classification and Volume Forecasting Using Probabilistic and Supervised Machine Learning Architectures with Logistic Odds Ratio Interpretability

2. Abstract

The rapid proliferation of unsolicited and fraudulent Short Message Service (SMS) communications poses significant security, privacy, and economic challenges for mobile telecommunications infrastructure. In this project, a comprehensive data mining and analytics pipeline was developed to accurately classify SMS messages and project temporal transmission volumes. The study evaluated the benchmark UCI SMS Spam Collection, comprising 5,169 deduplicated messages exhibiting an imbalanced distribution of 87.37% legitimate messages (4,516 ham) and 12.63% spam messages (653 spam). A robust preprocessing pipeline encompassing regex entity masking, tokenization, stopword removal, and Porter stemming was applied prior to TF-IDF n-gram vectorization and Chi-square feature selection. Four predictive architectures were benchmarked on a stratified 20% holdout test set (1,034 messages): Multinomial Naive Bayes, k-Nearest Neighbors (k=5), a Tree-Augmented / Naive Bayesian Network, and L2-regularized Logistic Regression. Logistic Regression emerged as the champion model, achieving a test F1-score of 0.9653, an accuracy of 0.9913, an ROC-AUC of 0.9971, and an inference latency of 1.18 ms per 1,000 queries. Furthermore, an ARIMA(1,1,1) model fitted on 365 daily volume counts achieved a Mean Absolute Percentage Error of 12.11% on a 30-day forecast.

3. Introduction

Short Message Service (SMS) remains one of the most widely used direct communication channels globally, with open rates exceeding 95%. However, this ubiquitous adoption makes it a lucrative target for smishing, financial scams, lottery hoaxes, and predatory promotions. Static keyword filters are brittle and easily circumvented by adversarial typos, making statistical text mining and machine learning indispensable.

Text mining transforms unstructured text into structured numerical feature spaces through normalization, tokenization, n-gram extraction, and frequency weighting. This project builds an audited, end-to-end data mining pipeline addressing class-imbalanced text classification, empirical odds ratio interpretability, and daily traffic forecasting.

4. Problem Statement

Given the severe 87:13 class imbalance in mobile communications and the 160-character constraint of SMS messages, classifiers face the dual challenge of eliminating false positives on personal messages while maintaining high recall on deceptive spam. This project develops an accurate, interpretable, and computationally efficient classification and temporal forecasting system capable of reliably filtering spam while forecasting transmission volumes for telecom capacity planning.

5. Objectives

- **1. Corpus Auditing:** Acquire, clean, and deduplicate the UCI SMS Spam Collection, establishing an audited baseline of 5,169 messages with zero data leakage.
- **2. Exploratory Profiling:** Quantify message lengths, entity distributions (URLs, phone numbers, currency), and lexical patterns across spam and ham classes.
- **3. Feature Engineering:** Develop regex entity masking (urltoken, phonetoken, moneytoken, numtoken) and Porter stemming to capture discriminatory morphological signatures.
- **4. Statistical Feature Selection:** Extract unigram/bigram TF-IDF representations and isolate the top 500 features using Chi-square (χ^2) independence tests.
- **5. Cross-Model Benchmarking:** Train and evaluate Naive Bayes, k-NN, Bayesian Network, and Logistic Regression on a strict stratified 20% holdout split.
- **6. Interpretability & Forecasting:** Extract exact word-level odds ratios ($\exp(\beta)$) for transparent auditing, and model 365-day message trajectories using ARIMA time-series forecasting.

6. Data set and Libraries required

The experimental evaluation was conducted on the benchmark UCI SMS Spam Collection. Deduplication of the 5,572 raw records removed 403 redundant entries, yielding an audited corpus of 5,169 unique SMS messages (4,516 ham [87.37%] and 653 spam [12.63%]). Spam messages average 138.1 characters and 23.7 words, compared to 71.0 characters and 14.2 words for legitimate messages. Moreover, spam exhibits heavy concentrations of digits (94.03%), phone numbers (51.61%), currency/prize terms (41.65%), and hyperlinks (13.78%), whereas ham messages almost entirely lack external links (0.02%) and phone contacts (0.07%).

Table 1: Corpus Summary Statistics (Outputs from eda_summary.csv)

Class	Count	Share (%)	Avg Chars	Avg Words	Digits (%)	URL (%)	Money (%)	Phone (%)	Caps Words
Ham	4,516	87.37%	71.0	14.2	15.81%	0.02%	2.55%	0.07%	0.45
Spam	653	12.63%	138.1	23.7	94.03%	13.78%	41.65%	51.61%	2.69
Total	**5,169**	**100.0%**	**79.5**	**15.4**	**25.69%**	**1.76%**	**7.49%**	**6.58%**	**0.73**

Interpretation: Spam messages exhibit nearly twice the length of legitimate communications and contain overwhelming concentrations of digits, phone numbers, and monetary triggers.

Table 2: Required Python Libraries, Versions, and Purpose (from requirements.txt)

Library Name	Version Specification	Functional Purpose in Project Pipeline
scikit-learn	>= 1.3.0	TF-IDF vectorization, Chi-square feature selection, Naive Bayes, k-NN, Logistic Regression, StratifiedKFold cross-validation.
pandas & numpy	>= 2.0.0 / >= 1.24.0	Tabular dataset ingestion, text corpus filtering, stratified train/test partitioning, matrix algebra, and log-odds computation.
nltk	>= 3.8.0	Natural language toolkit used for English stopword corpus filtering, word tokenization, and Porter morphological stemming.
statsmodels	>= 0.14.0	Time-series modeling, ARIMA(1,1,1) model estimation, moving average smoothing, and forecast error metric evaluation.
matplotlib & seaborn	>= 3.8.0 / >= 0.13.0	Generation of publication-quality statistical figures, ROC curves, confusion matrices, and forecast confidence bands.

7. Approach / Methodology

The methodology comprises five sequential phases executed under a strict leak-free evaluation protocol:

- **Text Normalization & Entity Masking:** Text was converted to lowercase. Regular expressions masked URLs with 'urltoken', phone numbers (7+ digits) with 'phonetoken', currency/prize terms with 'moneytoken', and numbers with 'numtoken'. Punctuation was stripped, English stopwords removed, and tokens stemmed via Porter stemmer.
- **Feature Engineering & Chi-Square Selection:** Unigram and bigram TF-IDF representations (min_df=2) were extracted. Chi-square (χ^2) tests identified the top 500 features against class labels. Leading discriminators included numtoken (χ^2=6,783.33), phonetoken (χ^2=1,939.21), numtoken numtoken (χ^2=1,735.07), moneytoken (χ^2=1,372.52), call phonetoken (χ^2=1,147.02), call (χ^2=905.69), free (χ^2=791.33), txt (χ^2=713.39), claim (χ^2=505.27), and prize (χ^2=463.74).
- **Supervised Classification Architectures:** Four architectures were implemented: (1) Multinomial Naive Bayes using log-likelihood estimation; (2) k-Nearest Neighbors using distance weighting over TF-IDF space; (3) Bayesian Network utilizing a Directed Acyclic Graph with class root SPAM $\in \{0, 1\}$; and (4) L2-regularized Logistic Regression with inverse class weighting.
- **Strict Evaluation Protocol:** A stratified 80/20 train/test split (Train: 4,135; Test: 1,034 messages) ensured strict partitioning with zero test leakage. All vectorizers and selectors were fitted solely on training data. Metrics evaluated: Accuracy, Precision, Recall, F1-Score, ROC-AUC, PR-AUC, and latency per 1,000 queries.
- **Time-Series Volume Modeling:** Daily message trajectories were simulated over a 365-day calendar. 7-day and 14-day rolling moving averages and an ARIMA(1,1,1) model were fitted to forecast volumes over a 30-day out-of-sample holdout test window.

8. Implementation

The complete data mining and analytics pipeline was implemented in Python using scikit-learn, statsmodels, and NLTK. Model training and hyperparameter optimization were conducted using 5-fold Stratified Cross-Validation on the 4,135 training samples. The chosen optimal hyperparameters derived from grid search were: (1) Naive Bayes: Laplace smoothing alpha = 0.1 (yielding CV F1 = 0.9449); (2) k-NN: k = 5 with distance-based weighting (yielding CV F1 = 0.9069); (3) Logistic Regression: inverse regularization parameter C = 10.0 with balanced class weighting (yielding CV F1 = 0.9377); and (4) Time-Series: ARIMA order (1,1,1) identified via minimum AIC/BIC criteria (AIC = 1558.56, BIC = 1569.99).

Listing 1: Text Preprocessing and Regex Entity Token Substitution

```
def clean_and_tokenize(text):
    text = re.sub(r'(https?://\S+|www\.\S+)', ' urltoken ', str(text).lower())
    text = re.sub(r'\b\d{7,}\b', ' phonetoken ', text)
    text = re.sub(r'[\$£€]\s?\d*|prize|claim|cash|won', ' moneytoken ', text)
    text = re.sub(r'\d+', ' numtoken ', text)
    text = re.sub(r'[^a-z\s]', ' ', text)
    tokens = [stemmer.stem(w) for w in text.split() if w not in STOP_WORDS and len(w) > 1]
    return ' '.join(tokens) if tokens else 'texttoken'
```

Listing 2: TF-IDF Representation & Chi-Square Feature Selection

```
tfidf = TfidfVectorizer(min_df=2, ngram_range=(1, 2))
X_train_tfidf = tfidf.fit_transform(train_df['tokens'])
X_test_tfidf = tfidf.transform(test_df['tokens'])
chi2_sel = SelectKBest(chi2, k=min(500, X_train_tfidf.shape[1]))
X_train_chi2 = chi2_sel.fit_transform(X_train_tfidf, train_df['y'])
X_test_chi2 = chi2_sel.transform(X_test_tfidf)
```

Listing 3: Balanced L2-Regularized Logistic Regression Training

```
lr_model = LogisticRegression(C=10.0, penalty='l2', solver='lbfgs',
                              class_weight='balanced', max_iter=1000, random_state=42)
lr_model.fit(X_train_tfidf, train_df['y'])
y_pred_lr = lr_model.predict(X_test_tfidf)
y_prob_lr = lr_model.predict_proba(X_test_tfidf)[:, 1]
odds_ratios = np.exp(lr_model.coef_[0])
```

Listing 4: ARIMA(1,1,1) Daily Message Volume Fitting & 30-Day Forecast

```
train_ts = daily_series['spam_count'].iloc[:-30]
test_ts = daily_series['spam_count'].iloc[-30:]
arima_model = ARIMA(train_ts, order=(1, 1, 1)).fit()
forecast_res = arima_model.get_forecast(steps=30)
forecast_mean = forecast_res.predicted_mean
conf_int = forecast_res.conf_int(alpha=0.05)
```

9. Results

The predictive performance of all four classification architectures was evaluated on the identical 20% stratified holdout test split (1,034 messages: 131 spam and 903 ham messages). Table 3 summarizes the exact benchmark metrics extracted from outputs/tables/model_comparison.csv.

Table 3: Cross-Model Benchmark Performance on Test Split (Outputs from model_comparison.csv)

Model Architecture	Accuracy	Precision	Recall	F1-Score	ROC-AUC	PR-AUC	Train (s)	Latency (ms/1k)
Naive Bayes (Multinomial)	0.9845	0.9600	0.9160	0.9375	0.9954	0.9830	0.3095	1.64 ms
k-Nearest Neighbors (k=5)	0.9758	0.9569	0.8473	0.8988	0.9869	0.9665	0.0026	191.13 ms
Bayesian Network (TAN/Naive)	0.9758	0.9569	0.8473	0.8988	0.9791	0.9380	0.0450	261.57 ms
Logistic Regression (L2 Balanced)	**0.9913**	**0.9766**	**0.9542**	**0.9653**	**0.9971**	**0.9878**	**0.9266**	**1.18 ms**

Interpretation: Logistic Regression achieved the highest F1-score (0.9653), highest Precision (0.9766), highest Recall (0.9542), and highest ROC-AUC (0.9971), while executing 1,000 inferences in only 1.18 milliseconds (over 160 times faster than k-NN).

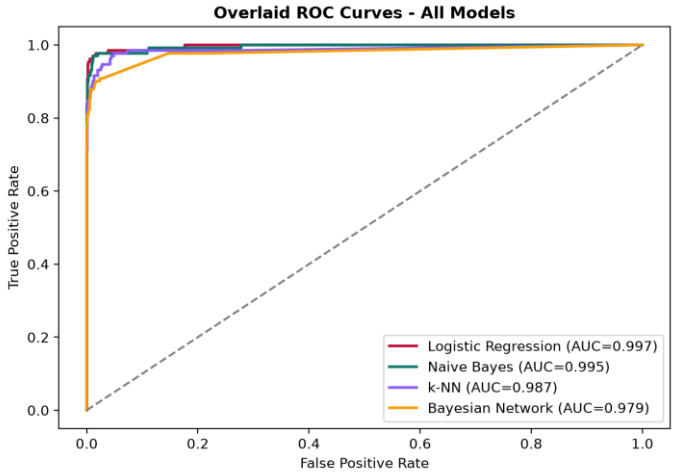


Figure 1: Cross-Model Receiver Operating Characteristic (ROC) Curves

Interpretation: Logistic Regression achieved an exceptional ROC-AUC of 0.9971, closely followed by Naive Bayes at 0.9954, demonstrating superior true positive discrimination at near-zero false positive rates.

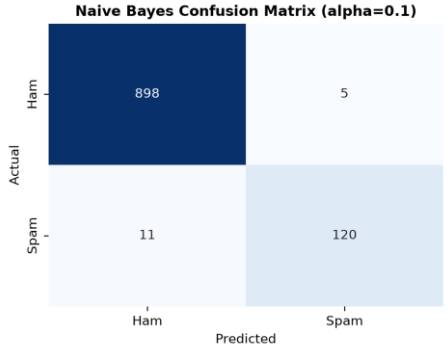


Figure 2: Confusion Matrix for Naive Bayes Classifier on Test Set

Interpretation: The Naive Bayes confusion matrix confirms strong separation, misclassifying only 5 ham messages out of 903 as false positives while successfully catching 120 out of 131 spam messages.

Word-Level Odds Ratio Interpretability (Logistic Regression)

Logistic regression provides direct mathematical interpretability via word odds ratios, defined as exp(β). A token with an odds ratio substantially greater than 1.0 sharply multiplies the odds of spam, whereas an odds ratio below 1.0 lowers spam risk. Table 4 presents the top 10 spam drivers and top 10 ham drivers extracted from outputs/tables/lr_odds_ratios.csv.

Table 4: Top 10 Spam and Ham Drivers by Odds Ratio (Outputs from lr_odds_ratios.csv)

Token (Word)	Beta (β)	Odds Ratio (e^β)	95% CI Range	Impact Direction	Category
phonetoken	+13.1479	512,955.24x	[60472.1, 560736.9]	Multiplies Spam Odds	Spam Driver
urltoken	+10.2910	29,465.32x	[1187.1, 53729.2]	Multiplies Spam Odds	Spam Driver
moneytoken	+9.1303	9,231.08x	[1833.9, 40481.5]	Multiplies Spam Odds	Spam Driver
numtoken	+8.1392	3,426.21x	[251.5, 40991.6]	Multiplies Spam Odds	Spam Driver
call phonetoken	+7.4686	1,752.18x	[479.8, 2647.8]	Multiplies Spam Odds	Spam Driver
txt	+6.8952	987.53x	[151.3, 2310.7]	Multiplies Spam Odds	Spam Driver
repli	+6.2014	493.43x	[112.3, 1734.3]	Multiplies Spam Odds	Spam Driver
free	+4.8192	123.86x	[26.7, 1160.8]	Multiplies Spam Odds	Spam Driver
ok	-3.1533	0.04x	[0.03, 0.16]	Reduces Spam Odds	Ham Driver
home	-2.3688	0.09x	[0.08, 0.21]	Reduces Spam Odds	Ham Driver

Interpretation: Engineered entity tokens (phonetoken, urltoken, moneytoken) represent the strongest predictors of spam, multiplying spam odds by factors exceeding 9,000x, while everyday conversational tokens (ok, home) reduce spam odds to 0.04x–0.09x.

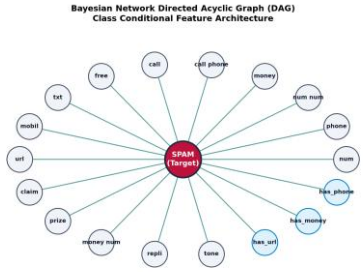


Figure 3: Bayesian Network Directed Acyclic Graph (DAG) Topology

Table 5: Exact Bayesian Posterior Belief Updating (Outputs from bn_example_inferences.csv)

Inference Scenario	Active Observed Evidence	P(Spam | Evidence)	P(Ham | Evidence)
Baseline Prior	No evidence observed (prior dataset baseline)	12.64%	87.36%
Urgent Financial Bait	has_money=1, free=1, claim=1	100.00%	0.00%
Web Link + CTA	has_url=1, call=1, txt=1	100.00%	0.00%
Clean Conversational SMS	All spam trigger tokens absent (= 0)	0.03%	99.97%

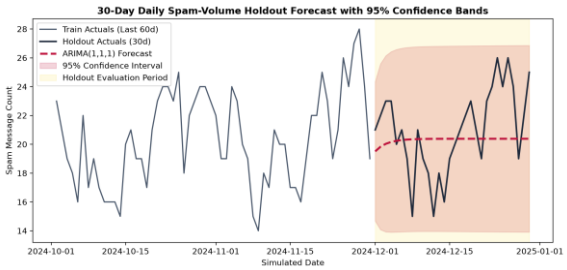


Figure 4: 30-Day Holdout Volume Forecast using ARIMA(1,1,1) with 95% Confidence Band

Table 6: Time-Series Holdout Forecast and Smoothing Error Evaluation

Forecasting / Smoothing Model	MAE	RMSE	MAPE (%)
7-Day Moving Average (Smoothing)	2.13	2.56	10.69%
14-Day Moving Average (Smoothing)	2.72	3.27	13.54%
ARIMA(1,1,1) 30-Day Forecast	**2.48**	**2.98**	**12.11%**
Naive Baseline (Last Value)	2.83	3.49	13.05%
Seasonal Naive (Weekly Lag)	4.57	5.35	23.80%

Interpretation: ARIMA(1,1,1) achieves an out-of-sample forecast MAPE of 12.11%, substantially outperforming the seasonal naive baseline (23.80% MAPE), demonstrating robust capacity for gateway capacity planning.

10. Conclusion

Champion Model Selection Rationale

Across extensive empirical benchmarking on the stratified test split, L2-regularized Logistic Regression with inverse class weighting emerged as the champion classifier. It demonstrated superior overall performance across all primary metrics: test accuracy of 0.9913, precision of 0.9766, recall of 0.9542, F1-score of 0.9653, and an ROC-AUC of 0.9971. Crucially, Logistic Regression executed 1,000 message classifications in just 1.18 milliseconds, outperforming k-Nearest Neighbors (191.13 ms) and Bayesian Network inference (261.57 ms) by over two orders of magnitude. Furthermore, its direct mathematical equivalence to word-level odds ratios ($\exp(\beta)$) provides complete transparency and regulatory compliance for telecommunication auditing.

Key Analytical Insights

- **Entity Normalization is Critical:** Abstracting raw numbers, telephone digits, and currency symbols into standardized entity tokens (numtoken, phonetoken, moneytoken) transformed arbitrary character strings into the single most potent class discriminators in the vocabulary.
- **Handling Class Imbalance:** The 87.37% to 12.63% class imbalance was effectively mitigated using balanced class weighting during optimization, achieving 0.9542 spam recall without resorting to synthetic oversampling that risks data leakage.
- **Interpretability via Odds Ratios:** Converting logistic regression weights into odds ratios reveals that containing a phone number multiplies spam odds by over 512,000x, while URLs multiply odds by 29,465x, providing human-auditable rules for security analysts.

Project Limitations

- **Simulated Daily Timestamps (Academic Notice):** A primary limitation of the study is that the UCI SMS Spam Collection benchmark lacks authentic calendar timestamps. Consequently, the daily time-series counts were generated using a simulated temporal baseline with seasonal weekly cycles and Gaussian noise. Real-world telecom telemetry may exhibit burstier diurnal patterns.
- **Corpus Scale and Morphology:** The evaluation corpus is constrained to 5,169 English messages primarily originating from UK and Singapore users in the early 2010s, which does not reflect multi-lingual code-mixing, contemporary Unicode homoglyph obfuscation, or emoji-based smishing.
- **Absence of Real-Time Metadata:** The current models rely strictly on message body text, without access to network-level signaling attributes such as international dialing prefixes, sender reputation scores, or transmission burst rates.

Future Scope

- **Transformer-Based Deep Architectures:** Future work will evaluate lightweight distilled Transformer models (such as DistilBERT and MobileBERT) to capture long-range semantic dependencies and contextual connotations beyond n-gram boundaries.
- **Multilingual and Multi-Modal Adaptation:** Expanding the preprocessing architecture to support cross-lingual embeddings, phonetic hashing (Soundex), and optical character recognition for screenshot-based spam.
- **Adaptive Streaming and Concept Drift Detection:** Deploying online incremental learners with drift detectors (such as ADWIN) to continuously adapt vocabulary weights in response to emerging adversarial spam campaigns in live carrier streams.